

Calidad de Software

Ingeniería de Software 4

Luz Mery Rios Alzate | Ingeniería de Sistemas

Toma de Decisiones en Ingeniería de Software Basada en CMMI y la ISO

Universidad Católica de Oriente
Vigilada Mineducación

Analizar, comprender y aplicar modelos de referencia como CMMI y las normas ISO para tomar decisiones informadas, justificadas y basadas en evidencia en proyectos de software, considerando calidad, riesgo y mejora continua.

Aspectos clave que puedes explorar:

- Modelos de referencia: CMMI e ISO.
- Procesos y áreas de proceso.
- Niveles de madurez y capacidad.
- Datos e indicadores para la decisión.
- Evaluación de alternativas.
- Gestión de riesgos en la toma de decisiones.
- Calidad del producto y del proceso.
- Cumplimiento de estándares.
- Trazabilidad y evidencia.
- Costo – beneficio.
- Contexto organizacional.
- Mejora continua.

Propósito de la actividad: Desarrollar la capacidad de tomar decisiones en proyectos de software utilizando criterios y prácticas de CMMI y la ISO, mediante un caso práctico, análisis de alternativas y justificación documentada.



Toma de Decisiones con CMMI y la ISO (Actividad en Equipo o Individual)

Cada estudiante deberá desarrollar un mini-taller o caso de análisis que incluya:

1 A. INTRODUCCIÓN

- ¿Cuál es la situación o problema de decisión?
- ¿Qué objetivos se quieren alcanzar?
- ¿Qué relación tiene con CMMI y la ISO?
- ¿Qué información es relevante para decidir?

2 B. MARCO DE REFERENCIA

- Conceptos clave de CMMI (niveles, áreas de proceso).
- Elementos relevantes de la ISO.
- Buenas prácticas relacionadas con la toma de decisiones.
- Cómo los modelos aportan al contexto del caso.

3 C. IDENTIFICACIÓN DE ALTERNATIVAS

- Definir al menos 2 o 3 alternativas de solución.
- Describir ventajas y desventajas de cada una.
- Considerar criterios como: calidad, costo, tiempo, riesgo, madurez de la organización.

4 D. ANÁLISIS Y EVALUACIÓN

- Aplicar un método de evaluación (matriz de decisión, AHP, ponderación, etc.).
- Usar datos e indicadores cuando sea posible.
- Relacionar la evaluación con prácticas de CMMI y la ISO.
- Mostrar la justificación del análisis.

5 E. DECISIÓN

- Seleccionar la alternativa más adecuada.
- Justificar la decisión tomada.
- Explicar cómo cumple con los principios de CMMI/ISO.
- Considerar impacto en el proyecto y en la organización.

6 F. PLAN DE IMPLEMENTACIÓN Y SEGUIMIENTO

- Definir acciones para implementar la decisión.
- Identificar responsables.
- Determinar indicadores. de seguimiento.
- Proponer mejoras continuas.

7 G. CONCLUSIONES Y APRENDIZAJE

- ¿Qué se aprendió del proceso de decisión?
- ¿Cómo ayudan CMMI y la ISO a tomar mejores decisiones?
- ¿Qué otros factores se deben considerar en el futuro?

Incluir en la entrega:

2 REFERENCIAS BIBLIOGRÁFICAS
(1 sobre CMMI y 1 sobre ISO).

1 ARTÍCULO CIENTÍFICO O TRABAJO DE GRADO
relacionado con toma de decisiones, CMMI o la ISO.

Importante: las fuentes deben estar correctamente citadas (APA). La decisión debe estar fundamentada en evidencia, no solo en opinión.

Métricas de Calidad de Software
Basadas en ISO y CMMI

2

Universidad Católica de Oriente
Vigilada Mineducación

Analizar, comprender y aplicar métricas de calidad de software utilizando como referencia las normas ISO (especialmente ISO/IEC 25010) y los modelos de proceso como CMMI, con el fin de evaluar indicadores de un proyecto y tomar decisiones para la mejora continua.

Aspectos clave que puedes explorar:

- Concepto y tipos de métricas.
- Métricas del producto y del proceso.
- Relación con la calidad del software (ISO).
- Indicadores en proyectos de software.
- Métricas y niveles de madurez (CMMI).
- Interpretación y análisis de datos.
- Uso de métricas para la toma de decisiones.
- Mejora continua basada en evidencia.

Propósito de la actividad: Analizar un conjunto de indicadores de un proyecto de software utilizando métricas de calidad, con base en ISO y CMMI, para identificar fortalezas, debilidades y proponer acciones de mejora.



ISO/IEC 25010

- Calidad del producto
- Características de calidad
- Subcaracterísticas
- Medición y evaluación

CMMI

- Medición y análisis (MA)
- Gestión de procesos
- Uso de indicadores
- Mejora continua

"Lo que no se mide, no se puede mejorar"

Métricas de Calidad de Software (Actividad en Equipo o Individual)

Cada estudiante deberá desarrollar un mini-taller o caso de análisis que incluya:

A. INTRODUCCIÓN
- ¿Qué son las métricas de calidad de software?
- ¿Por qué son importantes en un proyecto?
- ¿Cómo se relacionan con ISO y CMMI?
- ¿Qué decisiones se pueden tomar con base en métricas?

B. MARCO DE REFERENCIA
- Calidad del software según ISO/IEC 25010.
- Procesos de medición en CMMI.
- Tipos de métricas: producto, proceso y proyecto.
- Ejemplos de indicadores comunes.

C. CASO DE ANÁLISIS
- Analizar un conjunto de datos de un proyecto (real o simulado).
- Calcular e interpretar indicadores de calidad (ej. densidad de defectos, cobertura de pruebas, tiempo de corrección).
- Identificar fortalezas y debilidades.

D. ANÁLISIS Y RESULTADOS
- Interpretar los resultados de las métricas.
- Comparar con valores de referencia o buenas prácticas.
- Identificar riesgos y oportunidades de mejora.

E. PROPUESTA DE MEJORA
- Proponer acciones para mejorar los indicadores de calidad.
- Justificar las acciones con base en ISO/CMMI.
- Establecer cómo se medirá el impacto.

F. CONCLUSIONES
- ¿Qué aprendiste del análisis de métricas?
- ¿Cómo ayudan las métricas a tomar mejores decisiones?
- ¿Qué otros indicadores serían relevantes?

G. PRODUCTO FINAL
- Documento o informe con el análisis.
- Tablas y gráficos de indicadores.
- Conclusiones y recomendaciones.
- Presentación de los resultados.

Incluir en la entrega:

- **2 REFERENCIAS BIBLIOGRÁFICAS** (1 sobre ISO y 1 sobre CMMI).
- **1 ARTÍCULO CIENTÍFICO O TRABAJO DE GRADO** relacionado con métricas de calidad.

Importante: las fuentes deben estar correctamente citadas (APA) y el análisis debe basarse en datos reales o en un caso simulado.

Parcial – Punto 2

- **Punto 2: Entrega de artefactos del proyecto (30%)**
  - Entregable Proyecto: Artefactos del proyecto, planner actualizado, bitácoras desde la última fecha entregada por el estudiante.
- **Plan de pruebas y casos de prueba (30%).**
  - Casos de prueba: Para esto necesito que me manden las instrucciones para usar la herramienta de prueba seleccionada.

Plan de Pruebas

La calidad de la prueba depende tanto de la planificación como de la precisión del diseño.

Planificar → Diseñar → Ejecutar → Registrar → Analizar → Mejorar

PLAN DE PRUEBAS

Organiza alcance, enfoque, recursos, criterios, riesgos, tareas y cronograma.

CASOS DE PRUEBA

Convierten requisitos en verificaciones reproducibles y objetivas.



Idea final: "El plan organiza las pruebas; los casos hacen posible ejecutarlas de forma controlada y objetiva."

Diseño de casos de prueba: ¿qué es un caso?

ISO/IEC/IEEE 29119-1 define el caso a partir de precondiciones, entradas y resultados esperados.

PRECONDICIONES

Estado necesario antes de ejecutar.
Ej.: usuario autenticado; carrito con productos.

ENTRADAS / ACCIONES

Datos y acciones que conducen la ejecución.
Ej.: dirección, medio de pago, confirmar pedido.

RESULTADO ESPERADO

Comportamiento verificable que debe producir el sistema.
Ej.: pedido creado con total correcto.

Caso de prueba = condiciones + datos/acciones + resultado esperado

Un buen caso debe ser trazable a un requisito y permitir una decisión objetiva: pasa / falla.

1. **Identificar requisito** — ¿Qué comportamiento debe verificarse?
2. **Identificar condiciones** — ¿Qué debe ser cierto antes de ejecutar?
3. **Elegir técnica** — ¿Qué técnica ayuda a cubrir el comportamiento?
4. **Definir datos** — ¿Qué entradas permiten verificarlo?
5. **Definir resultado** — ¿Qué debe ocurrir y cómo se evidencia?

A la Verdad, por la fe y la ciencia

Un requisito puede generar varios casos

La cobertura no consiste en repetir el mismo escenario: debemos explorar condiciones relevantes.

REQUISITO

"El sistema permite registrar un pedido cuando los datos obligatorios son válidos."

→

CP-01
Entrada: Todos los datos válidos
Esperado: Pedido registrado

CP-02
Entrada: Dirección vacía
Esperado: Mensaje de validación

CP-03
Entrada: Medio de pago inválido
Esperado: No se crea el pedido

CP-04
Entrada: Campo obligatorio en límite
Esperado: Comportamiento definido

Referencias:

- Kaner, C., Bach, J., & Pettichord, B. (2002). Lessons Learned in Software Testing. Wiley. ISBN: 978-0471081128.
- Myers, G. J., Sandler, C., & Badgett, T. (2011). The Art of Software Testing (3rd ed.). Wiley. ISBN: 978-1118031964.
- Jorgensen, P. C. (2013). Software Testing: A Craftsman's Approach (4th ed.). Auerbach Publications. ISBN: 978-1466560680.
- Crispin, L., & Gregory, J. (2009). Agile Testing: A Practical Guide for Testers and Agile Teams. Addison-Wesley. ISBN: 978-0321534460.
- Graham, D., Black, R., & van Veenendaal, E. (2012). Foundations of Software Testing: ISTQB Certification (3rd ed.). Cengage Learning. ISBN: 978-1408044056.

Referencias:

- Whittaker, J. A. (2009). Exploratory Software Testing. Addison-Wesley. ISBN: 978-0321636416.
- Kinsbruner, E., Craske, A., & Feloney, S. (2019). Continuous Testing for DevOps Professionals. Apress. ISBN: 978-1484253505.
- Weinberg, G. M. (2008). Perfect Software and Other Illusions About Testing. Dorset House. ISBN: 978-0932633699.
- Whittaker, J. A. (2002). How to Break Software: A Practical Guide to Testing. Addison-Wesley. ISBN: 978-0201796193.
- Black, R. (2012). Advanced Software Testing - Vol. 1: Guide to the ISTQB Advanced Certification as an Advanced Test Analyst (2nd ed.). Rocky Nook. ISBN: 978-1937538497.

UCO
Universidad Católica de Oriente
Vigilada Mineducación

ALTA CALIDAD



¡Gracias!